UNIT 4 60 min

Conditionals: Making Decisions

if, elif, and else; indentation as structure; nested decisions; the order of branches.

By the end of this unit you will be able to

- Write an `if` statement and explain why indentation matters
- Choose between a chain of `elif` branches and separate `if` statements
- Trace which branch runs for a given input
- Nest conditionals and keep them readable
- Avoid the common mistakes: `=` for `==`, missing colons, and wrong branch order

The if statement

An `if` statement runs a block of code only when its condition is True. The condition ends with a colon. The block under it is indented, usually by four spaces. Python uses indentation to know where the block ends.

if_basic.py

```
temperature = 92

if temperature > 85:
    print("Turn on the fan.")
    print("Drink some water.")

print("Done checking.")
```

The last line is not indented, so it runs no matter what the temperature is.

if, elif, else

Use `elif` ("else if") to test a list of alternatives. Python checks them in order and runs **only the first** branch that is True. Then it skips the rest. The `else` branch catches everything left over.

grades.py

```
score = 84

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"

print("Grade:", grade)
```

Trace: which branch runs? TRACE

```
score = 84

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"

print("Grade:", grade)
```

Step	Line run	Variables after the line	Output so far
1	1	`score = 84`	—
2	3	`score = 84`	—
3	5	`score = 84`	—
4	6	`score = 84, grade = 'B'`	—
5	12	`score = 84, grade = 'B'`	`Grade: B`

Watch out

Order matters. If you test `score >= 70` first, a score of 95 lands in the "C" branch. Put the strictest condition first, or use separate `if` statements.

Nested decisions

You can put a conditional inside another one. Each extra level adds four spaces. If you find yourself nesting three or more levels deep, consider a different structure, such as checking for the failing case first and returning or continuing early.

nested.py

```
age = 17
has_ticket = True

if has_ticket:
    if age >= 18:
        print("Enter, adult.")
    else:
        print("Enter, minor. Guardian required.")
else:
    print("Buy a ticket first.")
```

Combining conditions

Use `and`, `or`, and `not` from Unit 3 to combine tests into one condition. This is often cleaner than nesting.

combined.py

```
age = 17
has_ticket = True

if has_ticket and age >= 18:
    print("Enter, adult.")
elif has_ticket:
    print("Enter, minor.")
else:
    print("Buy a ticket first.")
```

Conditional expressions

For a simple choice between two values, Python has a one-line form: `value_if_true if condition else value_if_false`. Use it for short values only.

ternary.py

```
n = 7
print("even" if n % 2 == 0 else "odd")
```

Predict the output THINK FIRST

Predict which lines print. Pay attention to indentation and branch order.

```
x = 5
if x > 3:
    print("big")
```

Answer: `big`

```
x = 5
if x > 3:
    print("a")
if x > 1:
    print("b")
```

Answer: `a b`
Two separate `if` statements are both checked.

```
x = 5
if x > 3:
    print("a")
elif x > 1:
    print("b")
```

Answer: `a`
Only the first true branch runs.

```
x = 0
if x:
    print("yes")
else:
    print("no")
```

Answer: `no`
0 is falsy.

```
print("even" if 4 % 2 == 0 else "odd")
```

Answer: `even`

Practice

▼ **Exercise 1 — Adult or minor**

Given `age = 16`, print `Adult` if the age is 18 or more, otherwise print `Minor`.

▼ **Exercise 2 — Ticket price**

A movie ticket costs $12. Students (age 17 and under) pay $8. Seniors (age 65 and over) pay $9. Given `age = 70`, print the price.

▼ **Hint**

Check the age groups in an order that makes each branch correct.

Discussion

1. Rewrite the grades example with separate `if` statements instead of `elif`. What breaks, and why?
2. When is nesting clearer than combining with `and`? When is it worse?
3. Give an example of a real-world rule that has an `else` branch you would not want to forget.

Key terms

Condition
An expression that evaluates to True or False and decides whether a block runs.

Branch
One path through a conditional. Each `if`, `elif`, and `else` defines a branch.

Indentation
Leading spaces that show which lines belong to a block. Python requires consistent indentation.

Block
A group of statements that run together, introduced by a colon and indented beneath a statement.

elif
Short for "else if". Tests another condition only if the previous ones were False.

Nested conditional
A conditional placed inside another conditional's block.

Conditional expression
A one-line choice written `a if condition else b`.

See all terms in the glossary →